Evaluating moving target defence against advanced persistent threat attackers modelled on documented intrusion campaigns

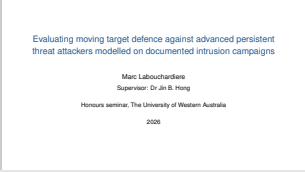
Marc Labouchardiere

Supervisor: Dr Jin B. Hong

Honours seminar, The University of Western Australia

2026

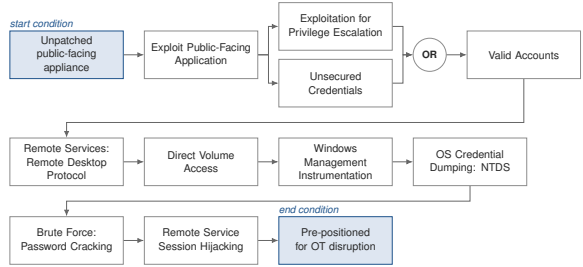
Evaluating moving target defence against advanced persistent threat attackers modelled on documented intrusion campaigns



- **0:00 to 0:15.** Name, supervisor.
- One line: “I test whether MTD’s measured effectiveness survives a change of attacker.”
- The footer is the talk’s timeline: six parts, one dot per slide.

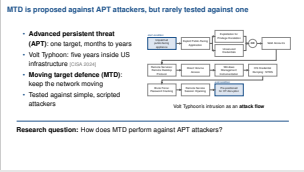
MTD is proposed against APT attackers, but rarely tested against one

- **Advanced persistent threat (APT)**: one target, months to years
- Volt Typhoon: five years inside US infrastructure [CISA 2024]
- **Moving target defence (MTD)**: keep the network moving
- Tested against simple, scripted attackers



Volt Typhoon's intrusion as an **attack flow**

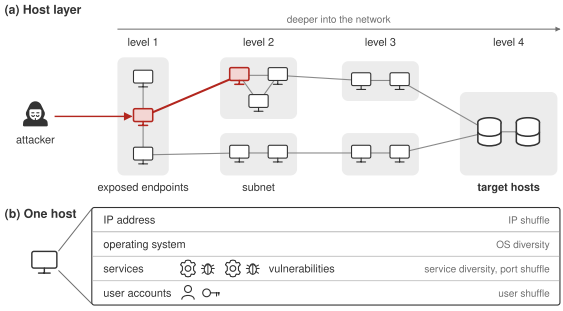
Research question: How does MTD perform against APT attackers?



- **0:15 to 1:30.** APT: a well-resourced attacker, often state-sponsored, pursuing one objective against one target over months or years (Alshamrani 2019). Espionage intrusions go undetected for a median of 122 days (Mandiant 2026).
- Volt Typhoon held footholds in US critical infrastructure for at least five years, signing in with stolen valid accounts. Its reconnaissance stayed valid because the networks never changed.
- MTD: change addresses, topology, software and credentials on a schedule, so what the attacker learned goes out of date even if it is never detected (Cho 2020).
- Point at the attack flow: “This is what a threat report gives us: the steps, in order, drawn by an analyst. It is the raw material of my attacker.”
- The gap: MTD is proposed against APTs, but tested in simulation against attackers far simpler than an APT (Jalowski 2026). Read the research question aloud.
- Transition: “To test that, I need an MTD simulator and an attacker.”

MTDSim: seven MTD mechanisms, one scripted attacker

- **MTDSim** [Brown 2023; Zhang 2023; Tay 2024]
- **Host layer**: where hosts are
- **Service layer**: what runs on them
- **Baseline attacker**: six **attack actions**, fixed order





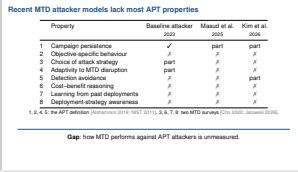
- **1:30 to 2:30.** The work I inherit: Brown built MTDSim at UWA, Zhang added simulated time, Tay added MTDShield, a reinforcement-learning agent that picks which mechanism to deploy.
- Panel (a): 50 hosts in four levels; the attacker enters at an exposed endpoint and works towards the target hosts.
- Panel (b): each part of a host has a mechanism that moves it. Seven MTD mechanisms, one deployed every deployment interval (200 s by default).
- **The two layers the results hinge on**: host layer = IP shuffle and two topology shuffles; service layer = port shuffle, OS diversity, service diversity. Plus user shuffle on credentials.
- Baseline attacker: scan host, enumerate host, scan port, exploit, brute force, scan neighbours; the same order every run.
- Transition: “Is a fixed procedure a fair stand-in for an APT?”

Recent MTD attacker models lack most APT properties

	Property	Baseline attacker 2023	Masud et al. 2025	Kim et al. 2026
1	Campaign persistence	✓	part	part
2	Objective-specific behaviour	✗	✗	✗
3	Choice of attack strategy	part	✗	✗
4	Adaptivity to MTD disruption	part	✗	✗
5	Detection avoidance	✗	✗	part
6	Cost–benefit reasoning	✗	✗	✗
7	Learning from past deployments	✗	✗	✗
8	Deployment-strategy awareness	✗	✗	✗

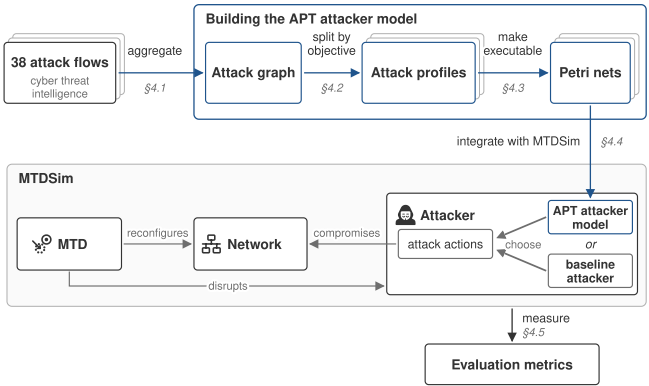
1, 2, 4, 5: the APT definition [Alshamrani 2019; NIST 2011]. 3, 6, 7, 8: two MTD surveys [Cho 2020; Jalowski 2026].

Gap: how MTD performs against APT attackers is unmeasured.



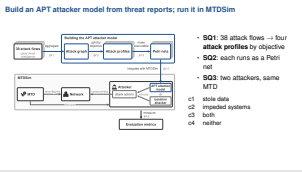
- **2:30 to 3:30.** Eight properties: four from what an APT is, four that two MTD surveys say attacker models lack. Jalowski et al. call the attacker model “the most glaring flaw in the MTD literature”.
- I scored the baseline attacker and the two most recent MTD evaluations that describe their attacker in Kill Chain or ATT&CK terms, the ones I expected to do best.
- Read the result: at most one property in full, campaign persistence, only for the baseline attacker. None has objective-specific behaviour.
- I targeted three: objective-specific behaviour, choice of attack strategy, adaptivity.
- Likely question: “How did you score them?” → backup *Scoring the eight properties*.

Build an APT attacker model from threat reports; run it in MTDSim



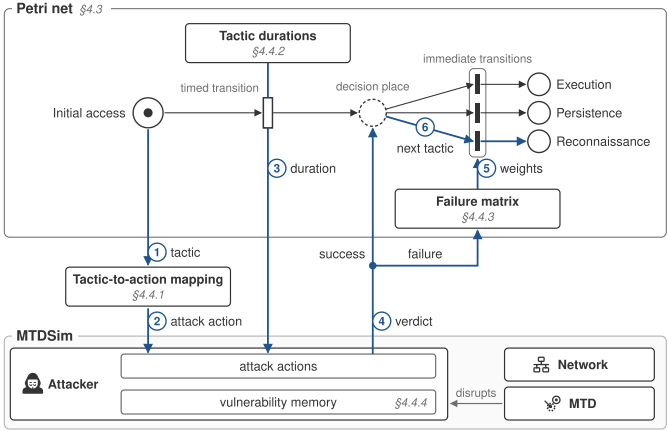
- **SQ1**: 38 attack flows → four **attack profiles** by objective
- **SQ2**: each runs as a Petri net
- **SQ3**: two attackers, same MTD

c1 stole data
c2 impeded systems
c3 both
c4 neither

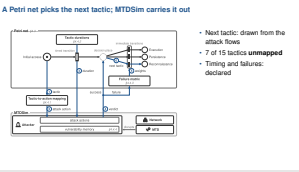


- **3:30 to 4:45.** Three sub-questions, one pipeline: point left to right along the top row.
- The 38 attack flows are analysts' hand-drawn graphs of reported intrusions, published by MITRE's Center for Threat-Informed Defense; the Volt Typhoon picture is one of the kind. Hand-drawn matters: automated extraction from report text is unreliable (best systems 80 % precision at 66 % recall on identifying techniques alone).
- Aggregated into an attack graph over the 15 ATT&CK tactics (what the attacker is trying to do: Initial Access, Lateral Movement, Exfiltration, . . .). Tactics, not techniques: 88 % of technique-to-technique edges come from a single attack flow.
- Split by two yes/no questions per attack flow: did it steal data, did it impede systems. c1 has 19 attack flows, c2 7, c3 7 (mostly ransomware), c4 5.
- Transition: "A graph does not run. Here is how it runs."

A Petri net picks the next tactic; MTDSim carries it out



- Next tactic: drawn from the attack flows
- 7 of 15 tactics **unmapped**
- Timing and failures: declared



- **4:45 to 6:15.** Walk the loop once with your finger, steps 1 to 6: the token's tactic leaves the Petri net; the tactic-to-action mapping turns it into an attack action; it runs for the tactic duration; MTDSim returns success or failure; after a failure the failure matrix reweights; the token moves on.
- Petri net: a generalised stochastic Petri net, the convention in MTD evaluation. Next tactic drawn in proportion to how many attack flows took that step, so runs differ.
- Tactic-to-action mapping: 8 of 15 tactics run one of MTDSim's six attack actions. The other 7 (e.g. Exfiltration, Stealth) are unmapped and only take time. The ceiling: MTDSim's attack actions only find and compromise hosts, so the model steals no data. Say this plainly.
- Threat reports record no timing and no failures. Tactic durations (up to 45 s on average) and the failure matrix are declared before any result, then varied. Example: a failed Initial Access returns to Reconnaissance with weight 0.75, up from 0.06.
- Likely questions: "Why these durations?" → backup *Tactic durations*; "Formal definition?" → backup *The Petri net*.

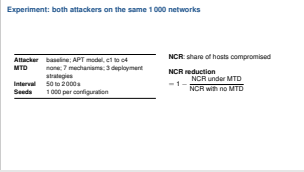
Experiment: both attackers on the same 1 000 networks

Attacker	baseline; APT model, c1 to c4
MTD	none; 7 mechanisms; 3 deployment strategies
Interval	50 to 2 000 s
Seeds	1 000 per configuration

NCR: share of hosts compromised

NCR reduction

$$= 1 - \frac{\text{NCR under MTD}}{\text{NCR with no MTD}}$$



- **6:15 to 7:00.** Keep this quick.
- The design choice that matters: every configuration runs on the same 1 000 seeds, so both attackers face the same 1 000 networks. Differences come from the attacker, not the network. 1 000 runs follows the recommendation for randomised algorithms (Arcuri and Briand).
- MTD: none; each of the seven mechanisms alone; random and alternative rotation; MTDShield as released. Intervals 50, 100, 200, 500, 1 000, 2 000 s.
- Targeted scenario: a run ends at the first target host compromised, or at 15 000 s.
- NCR, network compromise ratio (Zhang 2023). NCR reduction compares each attacker with its own runs with no MTD: 0 = no effect, 1 = nothing compromised.
- MTD settings ranked by the Scott–Knott effect-size test; the two rankings compared with Spearman's ρ.

No MTD: the APT attacker model acts more like an APT, but compromises less

	APT attacker model	Baseline attacker
Distinct openings (first 8 steps)	179 to 947	3
Attack actions per minute	0.68 to 1.23	1.71
Not flagged by a port-scan rule	89 to 98 %	82 %
Runs reaching a target host (ASP)	0.04 to 0.14	0.66
Share of hosts compromised (NCR)	0.15 to 0.21	0.50

Same cause for both halves: its tactic durations and the order of its tactics.

No MTD: the APT attacker model acts more like an APT, but compromises less

	APT attacker model	Baseline attacker
Distinct openings (first 8 steps)	179 to 947	3
Attack actions per minute	0.88 to 1.23	1.71
Not flagged by a port-scan rule	89 to 98 %	82 %
Runs reaching a target host (ASP)	0.04 to 0.14	0.66
Share of hosts compromised (NCR)	0.15 to 0.21	0.50

Same cause for both halves: its tactic durations and the order of its tactics.

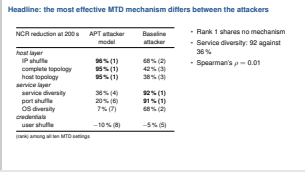
- **7:00 to 8:15.** First, no MTD at all: does the model behave differently?
- Row 1, choice of attack strategy: the baseline attacker's 1 000 runs start the same way; the attack profiles' runs part ways by the second to fourth step. Ranges are over c1 to c4.
- Rows 2 and 3, detection avoidance by proxy: it acts less often, and Snort's default port-scan rule (5 targets in 60 s) would flag fewer of its attack actions. MTDSim has no IDS; say so.
- Objective-specific behaviour (not on the slide): Exfiltration steps only in c1 and c3, Impact mostly in c2 and c3.
- Bottom rows: it reaches a target in far fewer runs. Pre-empt the objection: "Yes, with no MTD it is the weaker attacker. ASP and NCR alone would rate it weaker for exactly what makes it APT-like. With the baseline attacker's times it closes about two thirds of the gap."
- Transition: "Two different attackers, the same networks. Now switch MTD on."
- Likely question: "Isn't it just weaker?" $\rightarrow$ backup *Why it compromises less*.

Headline: the most effective MTD mechanism differs between the attackers

NCR reduction at 200 s	APT attacker model	Baseline attacker
host layer		
IP shuffle	**96 % (1)**	68 % (2)
complete topology	**95 % (1)**	42 % (3)
host topology	**95 % (1)**	38 % (3)
service layer		
service diversity	36 % (4)	**92 % (1)**
port shuffle	20 % (6)	**91 % (1)**
OS diversity	7 % (7)	68 % (2)
credentials		
user shuffle	−10 % (8)	−5 % (5)

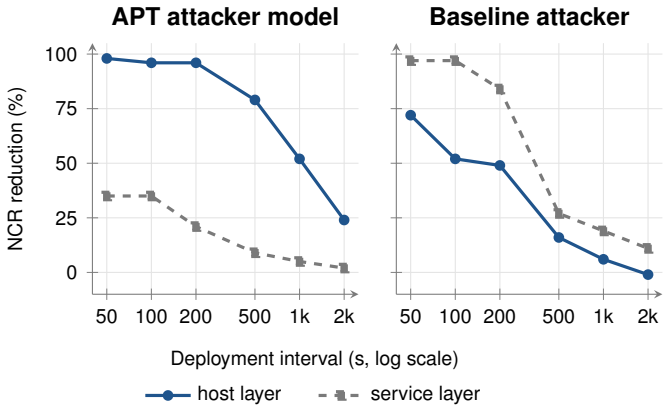
(rank) among all ten MTD settings

- Rank 1 shares no mechanism
- Service diversity: 92 against 36 %
- Spearman's $\rho = 0.01$



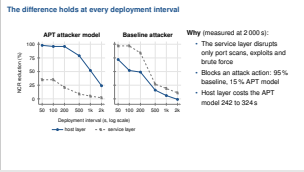
- **8:15 to 9:45. This is the result. Slow down.**
- Read the bold cells: against the APT attacker model, the host layer ranks first; against the baseline attacker, the service layer. Rank 1 shares no mechanism.
- The one number: service diversity removes 92 % of the baseline attacker's NCR and 36 % of the APT attacker model's, on the same networks, with the same deployments.
- $\rho = 0.01$: the two rankings are uncorrelated. Say "do not agree", not "reversed".
- APT attacker model averaged over c1 to c4.
- Only if asked: user shuffle is negative against the APT attacker model because it redraws accounts but leaves valid the ones the attacker holds, so each deployment is a fresh chance to log in.
- Transition: "Is that just the default interval?"

The difference holds at every deployment interval



Why (measured at 2 000 s):

- The service layer disrupts only port scans, exploits and brute force
- Blocks an attack action: 95 % baseline, 15 % APT model
- Host layer costs the APT model 242 to 324 s



- **9:45 to 11:15.** Left: in each panel, which line is on top? APT attacker model: host layer, at every interval. Baseline attacker: service layer, at every interval. Each line is the mean over the layer's mechanisms.
- The host layer keeps at least half of the APT attacker model's NCR off up to 1 000 s; the service layer does that for the baseline attacker only up to 200 s.
- Why: a deployment disrupts an attacker only if it lands while the attacker is doing what that layer breaks. The baseline attacker scans ports, exploits and brute-forces back to back; the APT attacker model sits mostly in unmapped tactics, which run no attack action.
- A host-layer deployment costs the APT attacker model 242 to 324 s: it has no host until its Petri net next reaches Lateral Movement. The baseline attacker loses none, because its procedure re-enumerates a host at once.
- Say "likely": the tactic durations and the order of the tactics most likely explain it; no run separates the two.
- Likely question: "Does the host layer just stop it getting in?" → backup *Footholds under the host layer*. Answer: largely yes, at short intervals.

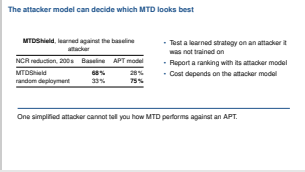
The attacker model can decide which MTD looks best

MTDShield, learned against the baseline attacker

NCR reduction, 200 s	Baseline	APT model
MTDShield	**68 %**	28 %
random deployment	33 %	**75 %**

- Test a learned strategy on an attacker it was not trained on
- Report a ranking with its attacker model
- Cost depends on the attacker model

One simplified attacker cannot tell you how MTD performs against an APT.



- **11:15 to 12:45.** The “so what” for anyone who evaluates MTD.
- MTDShield was trained and tested by Tay against the baseline attacker. It deploys service diversity in 70 % or more of its decisions against either attacker. Against the APT attacker model, random deployment does almost three times better. Most likely it learned the mechanism that disrupts its training attacker: a test in the training environment is a test on training data.
- Fair to Tay: it runs as released, outside its training conditions; this bounds MTDShield as released, not retrained.
- Recommendation 1: test a learned deployment strategy against an attacker model it was not trained on, beside random deployment over the same mechanisms.
- Recommendation 2: report each ranking with the attacker model behind it; compare across studies only on a shared attacker model.
- Recommendation 3, cost: to halve NCR, service diversity deploys every 1 000 s against the baseline attacker, but every 100 s against the APT attacker model: ten times the overhead.
- Optional question to the room: “If you were choosing an MTD for a real network, which

Limitations

- One simulator, one network size, no deployment cost
- Six attack actions: no data theft, no IDS
- Declared durations and failure matrix
- 38 attack flows
- Cause not isolated

Limitations

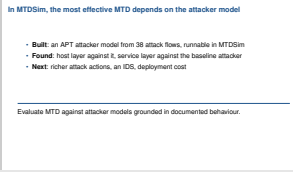
- One simulator, one network size, no deployment cost
- Six attack actions: no data theft, no IDS
- Declared durations and failure matrix
- 38 attack flows
- Cause not isolated

- **12:45 to 13:45.** State these yourself before anyone asks; examiners mark you for it.
- One simulator: every claim holds in MTDSim, on its default 50-host network, with deployments that cost nothing.
- MTDSim's six attack actions cap the model: 7 of 15 tactics unmapped, so no data theft, no impact, and no IDS to evade.
- Declared values: tactic durations and failure matrix are judgement where threat reports are silent, fixed before any result. Only the 45 s duration moves NCR much, and the ranking holds from a quarter to four times it.
- Small corpus: 38 attack flows, 18 attributed to a known group; three attack profiles rest on 5 to 7.
- Cause not isolated: tactic durations or the order of tactics. MTDSim's disruption rules were written for the baseline attacker; an APT holding a host through Command and Control might survive an IP shuffle. Both attackers meet the same rules, so the comparison is fair.
- The headline does not say "MTD fails against APTs" or "my attacker is the realistic one". It says the ranking *can depend* on the attacker model.

In MTDSim, the most effective MTD depends on the attacker model

- **Built**: an APT attacker model from 38 attack flows, runnable in MTDSim
- **Found**: host layer against it, service layer against the baseline attacker
- **Next**: richer attack actions, an IDS, deployment cost

Evaluate MTD against attacker models grounded in documented behaviour.



- **13:45 to 14:45.** Answer the research question in one sentence, then the two contributions. No new numbers; the audience has 92 against 36 %.
- Contribution 1: the APT attacker model, which any MTDSim study can run in place of the baseline attacker. Two of the eight properties in full and three in part, against at most one in full for three recent models. A proof of concept, built, run and open-sourced; not a claim that it reproduces APT behaviour.
- Contribution 2: the finding. At every deployment interval, the host layer disrupts the APT attacker model most and the service layer the baseline attacker.
- Future work: attack actions for actions on objectives, and an IDS; separate the tactic durations from the order of tactics; deployment cost; other networks; MTDShield retrained against the APT attacker model.
- Then move to the questions slide.

Questions?

How does MTD perform against APT attackers?

In MTDSim, it depends on which attacker it is tested against.

Marc Labouchardiere • Supervisor: Dr Jin B. Hong

Code and experiments: github.com/marc-la/MTDSim



- Leave this up during questions.
- Q&A map: weaker attacker → *Why it compromises less*; footholds → *Footholds*; durations → *Tactic durations*; components → *Ablations*; per profile → *Each attack profile*; scoring → *Scoring*; Petri net → *The Petri net*; MTDShield → *MTDShield as run*; ASP → *Disruption and ASP*; earlier numbers → *Verification*.

Backup: why the APT attacker model compromises less with no MTD

- Unmapped tactics take 34 to 57 % of an attack profile's run, and 15 to 30 % of its attack actions fail because their preconditions are missing (e.g. exploit before scan port)
- Rerun with the baseline attacker's times in place of its tactic durations, same order of tactics: ASP rises from 0.09 to 0.45, against the baseline attacker's 0.66, closing 63 % of the gap
- The order of its tactics most likely accounts for the rest
- The rankings compare each attacker with its own NCR with no MTD, so the lower starting point is unlikely to decide which mechanism ranks first

Dissertation Section 6.1, Appendix G.3, threat T2. Appendix G analyses were chosen after the results were seen (T9).

Backup: footholds under the host layer

- Under the three host-layer mechanisms at 50 to 200 s, 70 to 89 % of the APT attacker model's runs compromise no host, against 3 to 20 % of the baseline attacker's
- With no MTD: 1 % and none
- So the host layer disrupts the APT attacker model mostly *before* its first foothold: after a host-layer deployment it has no host until its Petri net next reaches Lateral Movement
- From 500 s the share falls for both attackers, and the host layer still ranks first against the APT attacker model

Dissertation Appendix G.4 (post hoc, T9); the claim stands on Chapter 5 without it.

Backup: are the declared tactic durations driving the result?

Mean duration (band)	low	declared	high
4.5 s (2.25 to 9)	0.165	0.163	0.159
35 s (17.5 to 70)	0.172	0.163	0.145
36 s (18 to 72)	0.170	0.163	0.148
45 s (11.25 to 180)	**0.267**	0.163	**0.059**

NCR with no MTD, each value moved alone, 400 runs per cell.

- Bands fixed before the run; one value moved at a time
- Only the 45 s (Persistence, Stealth, Command and Control) moves NCR much: Command and Control is the only tactic that finds new hosts
- At a quarter and four times the 45 s, rank 1 still shares no mechanism with the baseline attacker's, and the host layer still ranks above the service layer ($\rho = 0.03$ and -0.03)
- Untested: interactions between values

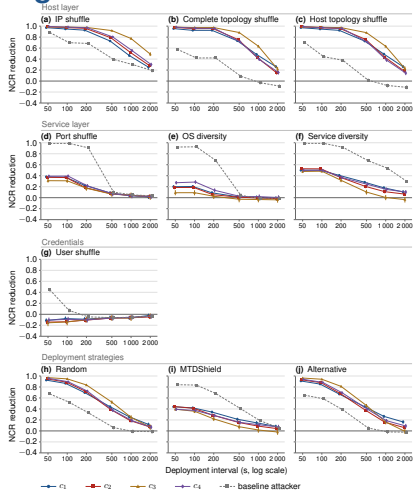
Dissertation Appendix D.1, threat T4.

Backup: what each component of the APT attacker model changes

Component removed	MTD	NCR with	without	Cohen's d
Attack profiles (use the whole attack graph)	no MTD	0.171	0.189	-0.28
	IP shuffle, 200 s	0.007	0.013	-0.47
	OS diversity, 200 s	0.159	0.174	-0.26
Failure matrix	no MTD	0.171	0.167	$+0.07$
	IP shuffle, 200 s	0.007	0.006	$+0.13$
Vulnerability memory	no MTD	0.171	0.169	$+0.04$
	service diversity, 200 s	0.109	0.106	$+0.07$

- Bold: not negligible ($|d| > 0.2$). The failure matrix changes *where* the attacker goes after a failure (back to Reconnaissance 76 to 83 % vs 8 to 10 %), not what it achieves
- Without the attack profiles, the same three host-layer mechanisms still rank first

Backup: each attack profile against the baseline attacker



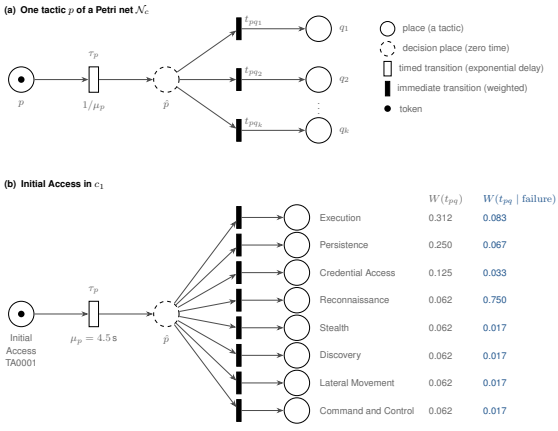
At 50 to 200 s the four attack profiles differ from one another less than the baseline attacker differs from the nearest of them (dissertation Figure 5.4).

Backup: scoring the eight properties

Attacker model	1 Pers.	2 Obj.	3 Strat.	4 Adapt.	5 Detect.	6 Cost	7 Learn.	8 Deploy.
Baseline attacker (2023)	✓	✗	part	part	✗	✗	✗	✗
Masud et al. (2025)	part	✗	✗	✗	✗	✗	✗	✗
Kim et al. (2026)	part	✗	✗	✗	part	✗	✗	✗
APT attacker model	✓	part	✓	part	part	✗	✗	✗

- Each property split into predicates; ✓ = every predicate true of the model's own behaviour; part = a behaviour predicate true, but set by a fixed rule or assumed
- One scorer, checked by blind scoring with language models (dissertation Appendix B)
- Adaptivity is in part because the failure matrix is a fixed rule of mine; detection avoidance in part because MTDSim has no IDS

Backup: the Petri net



Generalised stochastic Petri net [Ajmone Marsan 1995], extended with the failure matrix F:

$$(P, T, I, O, W, M_0, F)$$

Base weight of the move from tactic p to q:

$$W(t_{pq}) = \frac{n(p,q)}{\sum_{q'} n(p,q')}$$

with $n(p,q)$ the attack flows that draw $p \to q$. After a failed attack action:

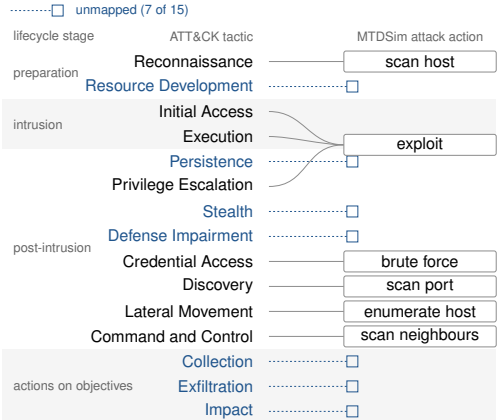
$$W(t_{pq} \mid \text{failure}) = \frac{W(t_{pq})\, F(p,q)}{\sum_{q'} W(t_{pq'})\, F(p,q')}$$

One token, starting on Reconnaissance.

Backup: tactic durations and the tactic-to-action mapping

Tactics	Mean (s)
Initial Access, Privilege Escalation, Credential Access, Lateral Movement (MTDSim's exploit)	4.5
Reconnaissance, Discovery (MTDSim's three scans)	35
Persistence, Stealth, Command and Control	45
Collection, Exfiltration, Impact	36
Execution, Defense Impairment	22.5
Resource Development	0

Last four rows: judgement, kept on the baseline attacker's time scale and under the 200 s default interval.

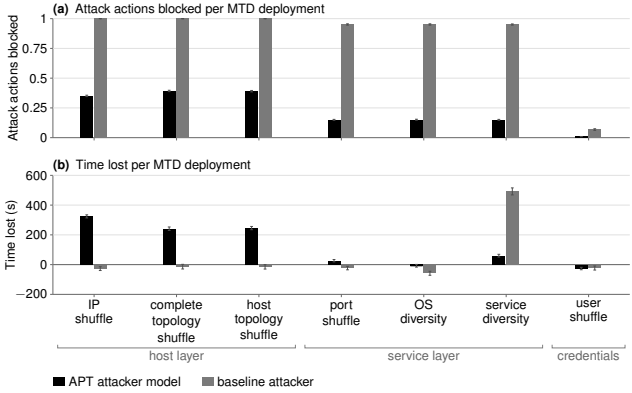


Backup: MTDShield as run

- Run as released by Tay (2024); correcting the release would make it my agent, not Tay's
- Trained at one deployment interval, against the baseline attacker, on 100-host networks
- Known limits of the release: published scores produced with exploration at one; four evaluation inputs read as zero; "do nothing" never a learning target
- Its detection input reads an unmapped tactic as "nothing detected"
- So the result bounds MTDShield *as released*, not MTDShield retrained against the APT attacker model

Dissertation Appendix E, threat T12.

Backup: disruption per MTD deployment, and ASP



- Measured at 2 000 s, because the next deployment caps time lost at the interval
- The baseline attacker's time lost is at or below zero after the first deployment: sent back to scan host, it gets a second attempt at its first host, as MTDSim intends
- On ASP reduction the layer order also holds against each attacker (Table F.3)

Backup: verification, and why no comparison with earlier MTDSim numbers

- An event tracer replays any run step by step, across Petri net, attacker and MTD
- MTDSim's code audited against what its papers describe; unit tests of the Petri net and the simulator
- Fixed seeds reproduce every run exactly
- Defects found in MTDSim were repaired before the evaluation, so results are not compared numerically with Zhang (2023) or Tay (2024)

Dissertation threat T3.

References

A. Alshamrani et al., “A survey on advanced persistent threats,” *IEEE Commun. Surveys Tuts.*, 21(2), 2019.

M. Ajmone Marsan et al., *Modelling with Generalized Stochastic Petri Nets*, Wiley, 1995.

A. Brown, T.-W. Lee and J. B. Hong, “Evaluating moving target defenses against realistic attack scenarios,” EnCyCriS, 2023.

Center for Threat-Informed Defense, “Attack Flow,” MITRE, v3.2.0, 2025.

J.-H. Cho et al., “Toward proactive, adaptive defense: A survey on moving target defense,” *IEEE Commun. Surveys Tuts.*, 22(1), 2020.

CISA et al., “PRC state-sponsored actors compromise and maintain persistent access to U.S. critical infrastructure,” Advisory AA24-038A, 2024.

Ł. Jalowski et al., “Rethinking the security assurances of MTD: A gap analysis for network defense,” *Future Internet*, 18(2), 2026.

M. Kim et al., “MTD in depth: Multi-phased moving target defense techniques against cyber-attacks based on cyber kill chain,” *FGCS*, 181, 2026.

M. T. Masud et al., “Vulnerability defence using hybrid moving target defence in Internet of Things systems,” *Computers & Security*, 153, 2025.

Joint Task Force, “Managing information security risk,” NIST SP 800-39, 2011.

C. Tantithamthavorn et al., “An empirical comparison of model validation techniques for defect prediction models,” *IEEE TSE*, 43(1), 2017.

J. K. Tay, “Using artificial intelligence to automate the deployment of moving target defence operations,” UWA research report, 2024.

W. Zhang, “Evaluating multiple moving target defence in the time domain,” Master’s thesis, UWA, 2023.